SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA14 COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 20%

QUESTIONS: Total Marks:100

Annexure A

Debugging

Code of Conduct:

I M Harinath/192425020(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student:

Q. No	Understanding of Debugging Concepts (25)	Error Identification(25)	Root Causes Analysis (20)	Error Corrections and Solution Design(20)	Testing and Validation(10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	

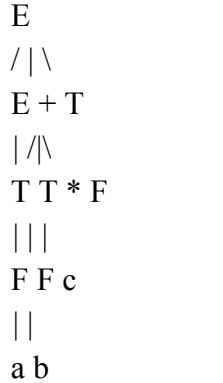
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	**/ 25**	**/ 20**	**/ 20**	**/ 10**	**/ 100**

Annexure A

Debugging

1. A syntax directed definition for the expression a + b * c generates an incorrect annotated parse tree because the semantic action for + is evaluated before the attributes of b * c are computed. Identify the error in the semantic rules and modify the rules so that the attributes are evaluated according to the correct dependency order and operator precedence.

Aim: To identify the error in the semantic rules and modify them so that the attributes are evaluated according to the correct dependency order and operator precedence.
Error Identification: The semantic action for the operator + is evaluated before the attributes of b * c are computed. As a result, the expression is interpreted as (a + b) * c instead of a + (b * c), which violates operator precedence.
Root Cause Analysis: The synthesized attribute of the multiplication node is not computed before the addition node is evaluated. Since multiplication has higher precedence than addition, the multiplication operation must be evaluated first.
Corrected Semantic Rules:
E → E1 + T { E.val = E1.val + T.val } E → T { E.val = T.val }
T → T1 * F { T.val = T1.val * F.val } T → F { T.val = F.val }
F → id { F.val = id.lexval }
Correct Annotated Parse Tree:



Testing and Validation: Assume the values:
a = 2 b = 3 c = 4
Then,
b * c = 3 * 4 = 12
a + (b * c) = 2 + 12 = 14
The expression is now evaluated in the correct dependency order and follows operator precedence.
Result: The semantic rules were corrected so that multiplication is evaluated before addition, producing the correct annotated parse tree and attribute evaluation order.

2. A compiler constructs an incorrect syntax tree for the expression a * (b + c) - d, where the + node is incorrectly placed as a child of the - node and the multiplication relationship is lost. Trace the tree construction during parsing, identify the faulty semantic action, and provide the corrected syntax tree construction rules.

Aim:
To identify the faulty semantic action responsible for the incorrect syntax tree and construct the correct syntax tree for the expression a * (b + c) - d.
Error Identification:
The + node is incorrectly attached as a child of the - node. Because of this, the multiplication relationship between a and (b + c) is lost, and the expression is interpreted incorrectly. Root Cause Analysis:
The parser creates the subtraction node before constructing the multiplication subtree. The semantic action does not preserve the grouping introduced by the parentheses, causing the + node to be attached directly under the - node.
Faulty Semantic Action:
E → E1 - T { E.node = newNode('-', E1.node, T.node) }
The multiplication subtree should be constructed before the subtraction node is created. Correct Syntax Tree Construction Rules:
E → E1 - T { E.node = newNode('-', E1.node, T.node) }
E → T { E.node = T.node }
T → T1 * F { T.node = newNode('*', T1.node, F.node) }
T → F { T.node = F.node }
F → (E) { F.node = E.node }
F → id { F.node = newLeaf(id.name) }
Correct Syntax Tree:

```
 -
 / \
 * d
 / \
 a +
 / \
```

b c
Testing and Validation:
The expression is evaluated in the following order:
b + c
a * (b + c)
(a * (b + c)) - d
The multiplication subtree is preserved correctly before subtraction is applied.
Result:
The faulty semantic action was corrected, and the syntax tree construction rules now generate the proper syntax tree for a * (b + c) - d, preserving operator precedence and parentheses.

3. A bottom up parser uses an S attributed definition to evaluate the expression x = (a + b) * c, but the synthesized attribute of the multiplication node is computed before the addition node is reduced. Identify the incorrect reduction or semantic action and rewrite the rules so that the syntax tree and synthesized attributes are correctly evaluated during bottom up parsing.

Aim:
To identify the incorrect reduction in the S-attributed definition and rewrite the semantic rules so that synthesized attributes are evaluated correctly during bottom-up parsing. Error Identification:
The synthesized attribute of the multiplication node is computed before the addition node is reduced. This causes the parser to evaluate multiplication before completing the addition operation inside the parentheses.
Root Cause Analysis:
In bottom-up parsing, reductions must occur from the leaves of the parse tree toward the root. The parser attempts to reduce the multiplication operation before the subtree representing (a + b) has been completely reduced.
Correct S-Attributed Definition:
S → id = E { S.node = assign(id, E.node) }
E → E1 + T { E.val = E1.val + T.val }
E → T { E.val = T.val }
T → T1 * F { T.val = T1.val * F.val }
T → F { T.val = F.val }
F → (E) { F.val = E.val }
F → id { F.val = id.lexval }
Correct Reduction Order:
a → F
b → F

F + F → E
c → F
E * F → T
id = T → S
Correct Syntax Tree:
=
/ \
x *
/ \
+ c
/ \
a b
Testing and Validation:
Assume:
a = 2
b = 3
c = 4
Then,
(a + b) = 5
5 * 4 = 20
x = 20
The synthesized attributes are evaluated in the correct bottom-up order.
Result:
The S-attributed definition was corrected so that the addition subtree is reduced before multiplication, resulting in the correct syntax tree and synthesized attribute evaluation.

4. An L attributed definition is used to translate the declaration int a, b, c, but during top down parsing the declared type int is assigned only to a and is not propagated to b and c. Identify the error in the inherited attribute propagation and modify the semantic actions so that the type information is correctly passed to every identifier.

To identify the error in inherited attribute propagation and modify the semantic actions so that the declared type is assigned to all identifiers.
Error Identification:
The inherited attribute containing the type int is assigned only to identifier a. The same type is not propagated to identifiers b and c during top-down parsing.
Root Cause Analysis:
The inherited attribute is not passed through the recursive identifier list. Therefore, only the first identifier receives the declared type while the remaining identifiers are left without type information.
Correct L-Attributed Definition:
D → T L { L.in = T.type }

T → int { T.type = int }
L → L1 , id { L1.in = L.in; id.type = L.in; }
L → id { id.type = L.in; }
Attribute Propagation:
T.type = int
L.in = int
a.type = int
b.type = int
c.type = int
Correct Parse Tree:
D
/ \
T L
| / | \
int L , c
/ | \
L , b
|
a
Testing and Validation:
After parsing the declaration int a, b, c, the symbol table becomes:
a : int
b : int
c : int
All identifiers correctly receive the type int.
Result:
The inherited attribute propagation was corrected so that the declared type int is passed to every identifier during top-down parsing.

5. A bottom up translation scheme for the expression a + b * c produces intermediate results in the order t1 = a + b and t2 = t1 * c. Identify the error in the attribute evaluation or semantic actions that causes the incorrect evaluation order. Modify the translation scheme so that multiplication is evaluated before addition while maintaining correct bottom up attribute evaluation.

Aim:
To identify the error in the bottom-up translation scheme and modify it so that multiplication is evaluated before addition while maintaining correct bottom-up attribute evaluation. Error Identification:
The generated intermediate code is:
t1 = a + b
t2 = t1 * c
This evaluates addition before multiplication and does not follow the correct operator

precedence.
Root Cause Analysis:
The semantic action for addition is executed before the multiplication subtree is reduced. In bottom-up parsing, multiplication should be reduced first because it has higher precedence than addition.
Correct Translation Scheme:
E → E1 + T { E.addr = newtemp(); emit(E.addr = E1.addr + T.addr); }
E → T { E.addr = T.addr; }
T → T1 * F { T.addr = newtemp(); emit(T.addr = T1.addr * F.addr); }
T → F { T.addr = F.addr; }
F → id { F.addr = id.name; }
Correct Bottom-Up Evaluation Order:
t1 = b * c
t2 = a + t1
Testing and Validation:
Assume:
a = 2
b = 3
c = 4
Then,
t1 = 3 * 4 = 12
t2 = 2 + 12 = 14
The generated intermediate code now follows the correct operator
precedence. Result:
The translation scheme was corrected so that multiplication is evaluated before addition while maintaining proper bottom-up attribute evaluation and generating the correct intermediate code.

RUBRICS FOR CALCULATION

Criteria	**Weigh tage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understandi ng of Debugging Concepts	25	Thorough understan din g; clearly identifies all key issues	Good understandi ng with most issues identified	Basic understan din g; some issues missed	Poor understan din g; problem not identified correctly
Error Identification	25	Effectively applies relevant concepts to analyze the	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts

		case deeply			
Root Causes Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Error Correction & Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Testing and Validation	10	Clear, well structured, and easy to understand	Organized and understandabl e presentation	Limited clarity and structure	Poor clarity; difficult to understand